

Laughter in the Wild: A Study into DoS Vulnerabilities in YAML Libraries

Shawn Rasheed

School of Fundamental Sciences
Massey University
Palmerston North
S.Rasheed@massey.ac.nz

Jens Dietrich

School of Engineering and Computer Science
Victoria University of Wellington
Wellington, New Zealand
jens.dietrich@vuw.ac.nz

Amjed Tahir

School of Fundamental Sciences
Massey University
Palmerston North
A.Tahir@massey.ac.nz

Abstract—YAML is a widely used serialisation language for data interchange and application configuration. Since its introduction, remote code execution vulnerabilities have been reported for YAML parsers, and countermeasures have been proposed. Even though denial-of-service (DoS) vulnerabilities affecting parsers for formats such as XML have been extensively studied, a similar investigation for YAML libraries is lacking. In this paper, we systematically study DoS vulnerabilities for 14 libraries for ten popular programming languages and as a result, we have discovered seven previously unknown vulnerabilities, which have been reported and are pending CVE identifiers.

Index Terms—Security, Denial of service, Vulnerability, YAML

I. INTRODUCTION

Deserialisation is a popular attack surface to exploit remote code execution and denial-of-service (DoS) vulnerabilities. The ability to customise deserialisation routines, specify code that can be executed during deserialisation through either the specification of types or code in the input stream has enabled these attacks. Language-agnostic deserialisation formats such as YAML, JSON and XML are generally seen as safer alternatives.

Among these formats, YAML (YAML Ain't Markup Language) [40] is widely used in data interchange, serialisation, and application/environment configuration. It is essentially a superset of JSON [15], with a lightweight syntax that is optimised for human readability and hand-editing. It has a type system comprised of scalars (numbers, strings and booleans) and collections (lists and maps.) Furthermore, it can support arbitrary types from the parser library's environment through "type tags". YAML is a cornerstone in IT environments where automation in development and deployment is largely driven by configuration files¹ that are either in YAML or JSON format. Examples of its use in this setting are for specifying API's according to the OpenAPI specification [19], Continuous Integration/Delivery (CI/CD) scripts, and describing cloud infrastructure as in Amazon's AWS CloudFormation templates [4].

Given its ubiquity, it is desirable that parsing YAML is safe and secure. However, this is not the case as several

remote code execution vulnerabilities in deserialising YAML data have been reported since 2013 for multiple languages, e.g. CVE-2018-13043 [27] and CVE-2017-18342 [26]. Remote code execution vulnerabilities in YAML parsing libraries have affected applications, examples being Swagger, Apache Brooklyn and Apache Camel [36] that use the SnakeYAML library [33]. The prevailing advice² is to use the safe load feature in these libraries as unsafe loading can be used to instantiate types with data from YAML input using its tag feature. Safe load limits the classes that are deserialised to the basic types available in YAML and rejects user-defined types from being deserialised.

There has been less attention on DoS attacks on YAML parsers. DoS attacks have been extensively studied for XML parsers [34]. In particular, vulnerabilities that result in exponential or quadratic blowups in resource usage while parsing input. Unlike DoS attacks based on flooding an application with network requests [1] or bugs that crash applications using vulnerabilities resulting from the violation of a programming language's semantics such as out-of-bounds access (e.g. buffer overflows) [5], these particular attacks involve a small input that triggers worst-case performance behaviour in code that results in the exhaustion of heap, stack memory or CPU resources.

The typical example is *billion laughs* [7] shown in Listing 1. The attacker uses a small XML document with multiple cross-referencing entities. Entity expansion by the parser is very expensive in terms of both memory and CPU consumption and this can be exploited to trigger a DoS attack. XML expansion results in large strings consisting of "lol" tokens, hence the name "billion laughs". In the listed XML code, the parser would expand to 10⁹ "lol" strings taking up nearly 3 gigabytes of memory. These attacks are also related to *algorithmic complexity attacks* [10] which aim at manipulating a system in a way so that the average-case performance of data structures deteriorates to worst-case.

Listing 1. XML Billion Laughs

```
<?xml version="1.0"?>
<!DOCTYPE doc [
  <!ENTITY l "lol">
```

¹https://www.theregister.co.uk/2018/11/19/popular_programming_language_yaml/

²https://security.openstack.org/guidelines/dg_avoid-dangerous-input-parsing-libraries.html

³<https://bitbucket.org/unshorn/dosyاملpub>

III. DENIAL-OF-SERVICE VULNERABILITIES

In the scope of libraries that process YAML, we look for methods that can be triggered while the parsed YAML structure is being converted to native data types and exhaust resources during this conversion.

One of the methods that is triggered in the construction of a map is the method to compute the hash value of a key. Maps are often implemented as hash tables, an efficient data structure that can look up a value that a particular key points to in $O(1)$ time. A hash function is used to compute a numeric hash value, which corresponds to an address location in memory that contains the associated value [8]. Since YAML supports complex keys, i.e., keys that are types other than plain numbers and strings, the conversion from YAML to the maps must trigger the computation of hash values for lists or maps that are used as keys. These computations can involve the recursive traversal of elements within them. For example, the hash value of a list may be computed based on the hash values of its elements.

To calculate the resource usage that is set off when such a method is triggered, we will use *call trees* that model a program's invocation of methods at runtime. The vertices in a call tree are method invocations, and two invocations ($inv1, inv2$) are connected by an edge if $inv2$ is the successor of $inv1$ on the stack at some stage during program execution. The *call tree* has the full calling context information since whenever a method is invoked, a new vertex is created.

A. Recursive Data

The obvious problem in computing hash values for containers is the possibility of recursive data. Dietrich et al. [12] discuss how deserialising recursive object graph topologies, object graphs with either cycles or loops, can lead to stack overflows. In the example in Listing 3, the key is a list with the sole element that is a reference to itself. As hash computation could be recursive on the elements in the list, the call tree for this computation is an infinite chain of method invocations that abnormally terminates from call stack exhaustion.

Listing 3. Loops and cycles in list as key

```
1 ? &id001
2 - *id001
3 : value
4 ---
5 ? &id001
6 - - *id001
7 : value
```

B. Hierarchical Data with Cross-references

Billion laughs-style attacks such as XML Entity Expansion (XEE) attacks and *SerialDoS* are enabled by hierarchical data with internal cross-references. In XML, it is the nested entities that is used for XEE attacks and in *SerialDoS*, a serialised nested object graphs with internal cross-references are used as deserialisation attack vectors. Listing 4 is the YAML equivalent of XML billion laughs implemented as nested lists. A similar expansion of the structure to a string

will occur when it is stringified (e.g. using the `toString()` method in Java).

Figure 1 shows the object graph that will result from processing the first YAML document in Listing 4. The anchor names correspond to the vertices in the graph. For clarity, we ignore internal structure within the objects in their language-specific implementations. The depth of this structure is two in this case. Note the number of objects created is seven, including the two strings "lol", and eight references.

When the list is converted to a string by triggering the outer list's `toString()` method, it sets off a recursive call to `toString()` on each of its children down the object graph. At runtime, this combination results in the call tree depicted in Figure 2. For brevity the `toString()` method is renamed to `m()` in the figure.

Whenever a new level is added to the object graph or rather the YAML list (i.e., the depth is increased from k to $k + 1$), each invocation `id_<k>A.toString()` triggers two new invocations `id_<k+1>A.toString()`, `id_<k+1>B.toString()`, and each invocation `id_<k>B.toString()` triggers two additional invocations `id_<k+1>A.toString()` and `id_<k+1>B.toString()`.

Evidently, the number of invocations is exponential in depth of the object graph. The first snippet, nested to a depth of two, expands to the list `[[[lol], [lol]], [[lol], [lol]]]` if the list is converted to a string. At depth three it doubles in size to `[[[[lol], [lol]], [[lol], [lol]]], [[lol], [lol]]]`.

In the second document in the listing, the nested list is used as a complex key with the value "value". As the insertion of the value into the map by the parser will require computing the hash value for the nested list, it will trigger the hash function on the outer list. The rest of the invocations are similar to the analysis for stringification. The notable difference is that the computation does not result in memory exhaustion because unlike the string computation which concatenates and expands the string with each invocation, the accumulated hash value is a fixed size integer.

Another key requirement is that the depth of the object graph must be limited to avoid a stack overflow yet deep enough to cause memory or CPU resource exhaustion.

Listing 4. Nested lists

```
1
2 - &id2A
3   - &id3A [lol]
4   - &id3B [lol]
5 - &id2B
6   - *id3A
7   - *id3B
8
9 ---
10 ? - &id2A
11   - &id3A [lol]
12   - &id3B [lol]
13   - &id2B
14     - *id3A
15     - *id3B
16 : value
```

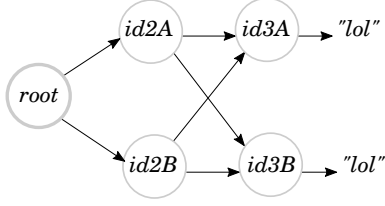


Fig. 1. Graph representation of list

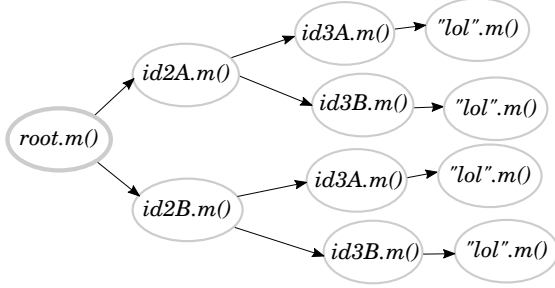


Fig. 2. Call tree

IV. EXPERIMENTAL SETUP

The experiment was conducted on an Intel i5 2.40 GHz machine with 16GB memory and Linux Ubuntu 18.04.1, within a Docker container to improve reproducibility. We generated test vectors, wrote drivers programs for each library, and the drivers were executed against each of the test vectors. Each execution was set to timeout after two minutes, and the exit code was recorded. Cases where the exit code was none-zero, i.e., the execution either timed-out or failed, were examined in more detail to determine the behaviour of the library.

A. Test Vectors

We assume that the attacker has access to an endpoint that accepts YAML data and uses the parser’s safe loading feature to deserialise the input.

We created nine test vectors designed to trigger DoS attacks. The nine test vectors as listed in Table I are based on two recursive data structures and a third with a deeply nested list. The two recursive data structures are a list with a reference to itself constituting a loop; a list nested within another with a reference to its parent forming a cycle. They are both as in Listing 3. The third is a deeply nested list as described in Section III. The character counts listed in the table show that each of the vectors are small at under ten kilobytes in file size at most and hence, YAML libraries should be expected to parse them reasonably fast (within seconds.) In the case of

TABLE I
CHARACTER COUNT FOR TEST VECTORS

	Loop	Cycle	Nested lists
Test vectors 1 (as key in map)	24	26	12,182
Test vectors 2 (as value in map)	21	23	11,783
Test vectors 3 (as element in list)	20	22	12,174

the deeply nested list, there are 98 references/aliases in the test vectors. The nesting depth was set at 50 exercising care that the depth would not exhaust the stack before triggering a potential long-running computation.

All three of the structures were duplicated as keys in a map, values in a map and as elements in a list to produce the nine test vectors. The test vectors are available in the repository⁴.

All nine files were generated using the Java SnakeYAML [33] library and the files were verified for correctness by parsing them with the LibYAML parser for C [17], which is the YAML reference parser implementation that forms the basis for many YAML libraries (e.g. Python YAML [29], Rust Yaml [31].)

B. Library Selection

We selected the YAML processing libraries from the list available at the official YAML website⁵. We confirmed the popularity of the libraries on package management repositories (e.g. NuGet⁶ for C#, npm⁷ for JavaScript and CPAN⁸ for Perl) to ensure that these libraries were the most widely used in their respective language communities. If multiple libraries exhibited similar popularity such as in the case of Perl, we included them. In some cases where data format parsers (e.g. Jackson for Java which depends on SnakeYAML for parsing YAML) depended on a library that is already in the list, we did not include it. We also excluded low level parser libraries such as LibYAML [17] from our study, as our focus is only on libraries that parse and deserialise YAML to their respective language’s native data types. We have also excluded libraries that only perform type-directed deserialisation, which is the case where users must specify the types of the objects in the stream (e.g. *serde-yaml* [32] and *go-yaml* [14], for Rust and the Go language.)

V. RESULTS

Tables III, IV and V lists a summary of the results from the experiment for the test vectors, *Loop*, *Cycle* and *Nested aliases*. *Success* indicates the driver terminated without errors, *Timeout* indicates that the driver timed-out after the threshold of two minutes. *Error* indicates that the driver exited with an error, which is parsing failure, and *Out of memory* if the driver abnormally terminated from a failure to allocate memory.

⁴<https://bitbucket.org/unshorn/dosyamlpub/src/master/src/testvectors/>

⁵<http://yaml.org>

⁶<https://www.nuget.org/>

⁷<http://www.npmjs.com>

⁸<https://www.cpan.org/>

TABLE II
ANALYSED LIBRARIES

Language	Library	Version	Lang. version
JavaScript	js-yaml	3.12.1	NodeJS 8.10.0
PHP	php-yaml	2.0.4	PHP 7.2.15
	Symfony Yaml	4.2.0	
Python	PyYAML	3.13	Python 3.6.7
	ruamel.yaml	0.15.89	
Dart	yaml	2.1.15	Dart VM 2.1.1
C#	YamlDotNet	5.2.1	Mono 4.6.2
Perl	YAML::Syck	1.31	Perl 5, version 26
	YAML::XS	0.76	
	YAML	1.27	
Ruby	psych	3.1.0	2.5.1
Java	SnakeYAML	1.23	OpenJDK 1.8.0_191
Rust	yaml-rust	0.4	Rust 1.32
Swift	YamlSwift	3.4.3	4.2.3

TABLE III
LIBRARY EXIT CODES FOR TEST VECTORS 1: STRUCTURE AS KEY IN MAP

Library	Loop	Cycle	Nested lists
JavaScript (Node)	Success	Success	Error
PHP PECL Yaml	Success	Success	Success
PHP Symfony	Error	Error	Error
Python YAML	Error	Error	Error
Python ruamel.yaml	Success	Error	Error
Dart	Stack overflow	Stack overflow	Timeout
C#	Stack overflow	Stack overflow	Timeout
Perl Syck	Error	Error	Success
Perl XS	Success	Success	Success
Perl YAML	Error	Error	Error
Ruby	Success	Success	Timeout
Java	Stack overflow	Stack overflow	Timeout
Rust	Success	Success	Out of memory
Swift	Error	Error	Timeout

TABLE IV
LIBRARY EXIT CODES FOR TEST VECTORS 2: STRUCTURE AS VALUE IN MAP

Library	Loop	Cycle	Nested lists
JavaScript (Node)	Success	Success	Success
PHP PECL Yaml	Success	Success	Success
PHP Symfony	Error	Success	Success
Python YAML	Success	Success	Success
Python ruamel.yaml	Success	Success	Success
Dart	Success	Success	Success
C#	Success	Success	Success
Perl Syck	Success	Success	Success
Perl XS	Success	Success	Success
Perl YAML	Success	Success	Error
Ruby	Success	Success	Success
Java	Success	Success	Success
Rust	Success	Success	Out of memory
Swift	Error	Error	Error

TABLE V
LIBRARY EXIT CODES FOR TEST VECTORS 3: STRUCTURE AS ITEM IN LIST

Library	Loop	Cycle	Nested lists
JavaScript (Node)	Success	Success	Success
PHP PECL Yaml	Success	Success	Success
PHP Symfony	Error	Success	Success
Python YAML	Success	Success	Success
Python ruamel.yaml	Success	Success	Success
Dart	Success	Success	Success
C#	Success	Success	Success
Perl Syck	Success	Success	Success
Perl XS	Success	Success	Success
Perl YAML	Success	Success	Error
Ruby	Success	Success	Success
Java	Success	Success	Success
Rust	Success	Success	Out of memory
Swift	Error	Error	Success

1) *JavaScript*: From the tests, the *js-yaml* [16] YAML library was executed in the NodeJS environment. It failed on the nested structure as a key test with the error: `RangeError: Invalid string length in the Array.toString method`. Further investigation revealed that *js-yaml*, converts keys (keys in the parsed YAML could be objects other than strings or numbers) to strings as JavaScript objects do not support non-string or non-numeric values in maps.

We investigated with nesting levels lower than 50, and with a structure of depth 30, we were able to crash NodeJS with an out of memory error confirming that DoS via memory exhaustion is possible for the library. The vulnerability has been reported to the npm registry for JavaScript packages and it has been fixed⁹. We also reproduced the same results on the browser with *js-yaml*.

2) *PHP*: The PHP Yaml Extension [24] succeeded the tests. Since PHP does not support non-string/numeric keys in its primary data structure, associative arrays, we found that it stringifies complex keys. In contrast to *js-yaml*'s approach, instead of the full string representation of the structure it uses PHP's default serialisation format for the key. As PHP's serialisation supports internal references [23] [11] this results in a compact representation if the key is a data structure that's compatible with PHP. If so, the compact key representation does not cause a time-out or memory exhaustion.

We experimented with variants of the YAML structure to create a new test vector that results in exponential growth of the serialised representation and we were able to cause memory exhausted for the PHP YAML Extension. The key idea is to introduce a list with multiple complex keys at each nesting level so that a composite key cannot be built by once-off serialisation but serialisation must be invoked at inner levels. E.g. `{[[{a}: a]]: b}` where the key is not a valid PHP structure and the inner key must be serialised before the outer key. The second library, PHP Symfony [28], that we studied does not support complex keys.

We have reported the issue for the PHP Yaml Extension and

⁹<https://www.npmjs.com/advisories/788>

it has been accepted as a security bug¹⁰, pending a CVE after a fix is available.

3) *Python*: Python does not support complex keys, as the language does not allow mutable structures such as lists and maps as keys in dictionaries. With this limitation the Python YAML libraries are not compliant with the YAML specification.

4) *Dart*: Dart successfully processes test vectors 2 and 3. On test vectors 1, structures as keys, Dart throws stack overflow errors for keys that have cyclic references and it timed out on the nested lists case.

5) *C#*: The results were similar to Dart. In the case of test vectors 1, the C# runtime exited with a stack overflow for the recursive structures case, and timed out for nested lists as a key.

6) *Perl*: The Perl XS [22] and Syck [21] libraries use a fixed length string representation for complex keys and does not cause DoS. Otherwise the errors were due to parsing issues.

7) *Java*: For test vectors 1, the Java driver reported a stack overflow for the recursive structures, and times out for the nested lists case on hash computation. We have reported the bug and it has been accepted as an issue¹¹ pending a fix.

8) *Ruby*: Ruby times out on the nested lists as key test while computing hashes. It does not cause a stack overflow as hash computation returns a special constant if recursion is detected.

9) *Rust*: The operating system kills Rust due to memory exhaustion in all three test vectors for the nested lists case.

10) *Swift*: The Swift YAML library causes errors in all but one test. The errors are due to parsing and in the nested lists as key case, it times out.

VI. DISCUSSION

Type-directed parsers and restrictions on YAML features offer a level of protection in the case of some libraries, although type-directed deserialisation might also not be completely impervious to attacks¹². It can be difficult to strike a correct balance between usability and security. Earlier DoS vulnerabilities, e.g. CVE-2012-1152 [25], have been due to invalid input. There is a discussion in the language-theoretic security community¹³ on avoiding such attacks by focussing on the correctness of parsers [18]. However, the vulnerabilities that inflict the libraries are not due to input that is invalid at the syntactic level, but due to inadequacies in hash function implementation: no guards against recursive data in the case of Java, Dart and C#. Standard libraries could implement hash functions for containers that use dynamic programming to avoid redundant computation of elements within them.

In string serialisation (`toString` in Java and JavaScript) there is no check on the unbounded accumulation of the

string representation of nested data structures like Arrays. In the .NET framework, collection classes do not override the `ToString` method¹⁴ from `Object` making string serialisation impossible by default, and leaving it to users to implement based on their specific use cases.

In the case of XML parsers, disabling inline document type definitions (DTD's) [35], which makes it impossible to define entities, is a protection against memory exhaustion while parsing user supplied XML. Another measure is to limit the expansion of recursively defined entities [35].

A. Countermeasures

Countermeasures for XML Entity Expansion attacks and quadratic blowup based on prohibiting entities or limiting the depth of nested entities are excessively restrictive. For Java serialisation, a filter mechanism described in JEP 290 [20] is available that can be configured to impose restrictions on the depth of input graph, total count of internal references and the input size. For the YAML parsers we studied, there are no restrictions on parsing other than disabling the alias feature, recursive data structures for the Ruby YAML library, and disabling type tags for the Python, Ruby and Java YAML libraries.

The approach we propose is a more granular filter that is more permissive than the available techniques. We attempt to estimate the size of the call tree and prevent further processing (conversion to native data structures) after parsing the YAML representation graph if the properties of the object graph suggests a call tree with an inviable execution time. This technique will result in an analysis that rejects fewer object graphs as malicious, i.e., it returns less false positives.

An approach such as rejecting inputs with references, reference counting or limits on graph depth rejects innocuous object graphs. To reject inputs with references is a very imprecise approach. A restriction on object graph depth is also not optimal as there are cases where deeply nested graphs are benign inputs. One example being serialised linked lists. In addition, the test vectors that caused DoS in our experiments are not excessively deep. Reference counting thresholds are of little help as we have demonstrated that test vectors where the resulting graph has as few as 98 internal references can cause resource exhaustion. With regard to setting a threshold in our approach, one perspective is to set a threshold that is guaranteed to execute in a few seconds, which will let in anything but malicious inputs. In this case there is no need to finely tune the threshold to a particular execution environment.

In a call tree, a call chain is a sequence of edges, $e_1, \dots, e_n$, where the target of e_i is the same as the source of e_{i+1} in the call tree. For *SerialDOS*, each call chain in the call tree is a path from the root of the graph to a leaf. The number of call chains executed is the number of *root-leaf* paths in the object graph. A filter can thus be setup to reject object graphs that can result in call trees that have call chain counts above a specified threshold.

¹⁴<https://docs.microsoft.com/en-us/dotnet/api/system.collections.arraylist?view=netframework-4.7.2#methods>

¹⁰<https://bugs.php.net/bug.php?id=77720>

¹¹<https://bitbucket.org/asomov/snakeyaml/issues/432/aggressive-yaml-anchors-causing>

¹²<https://github.com/dtolnay/serde-yaml/issues/53>

¹³<http://langsec.org/>

1) *Algorithms for Path Counting:* For a directed acyclic graph, $G(V, E)$ where V and E are the sets of vertices and edges respectively, the number of simple paths between two vertices can be counted using either memoisation and depth-first search with a runtime of $O(|E| + |V|)$, or by topologically sorting the graph and adding the number of paths arriving to each vertex from its predecessors [38]. In the memoisation case, the number of paths is given by the recurrence equation:

$$\text{Paths}(s, d) = \begin{cases} \text{if } s = d & = 1 \\ \text{otherwise} & = \sum_{v \in \text{Successors}(s)} \text{Paths}(v, d) \end{cases}$$

2) *Implementation:* We implemented a proof of concept in C++ (source code available in the repository¹⁵) using the LibYAML parser. It first converts a YAML document to a directed acyclic graph that captures objects (lists, anchors and mappings) as vertices and containment of any of these objects, either as an alias or a concrete object within another, as an edge. If the input has cycles the graph can be condensed to an acyclic graph using strongly connected components [37]. In our implementation the number of paths from the root to the leaves is computed using depth-first search and memoisation to avoid exponential runtime. If the number of paths exceed a user-defined threshold it can be rejected as unsafe to deserialise. We tested our prototype on the test vectors with nested list structures on hardware with the same configuration as our previous experiments, and it checks the input in an average of 27ms. Therefore, this technique could be used as an effective precondition check for parsers to reject potentially malicious input.

VII. RELATED WORK

A. Billion Laughs

Dietrich et al. [12] discuss billion laughs for Java deserialisation and how it affects other languages. Namely, Ruby, C# and JavaScript. One of the contributions in their study is adapting Coekaerts' *SerialDoS* [6] to cause memory exhaustion.

Späth et al. [34] have conducted a systematic evaluation of XML parsers in different languages and studied if they are vulnerable to the well-known XML entities attack and quadratic blowup in parsing. They also discuss recursively defined entities, which reference each other in their definitions. Even though the XML specification forbids such definitions, some parsers are susceptible to DoS attacks via such XML documents that put the parser in an infinite loop. This attack is similar in nature to the turtles vulnerability in the Dietrich et al. study [12].

B. Algorithmic Complexity Vulnerabilities

Crosby et al. [10] report a hash collision vulnerability in Perl's implementation of associative arrays/hash tables, which resulted in inserting many keys with the same hash code into a hash table, triggering worst-case performance, $O(n)$

rather than the typical-case performance, $O(1)$. As hash tables are used to process HTTP requests this can be used as an attack vector on vulnerable hash table implementations. YAML and JSON could also be used as attack vectors. The issue was reported for multiple programming languages by Julian Wälde and Alexander Klink [2]. In the case of Java, `String.hashCode()` hash function was not sufficiently collision resistant. As `hashCode()` value is used in the implementations of `HashMap` and `Hashtable` classes it can be used in attacks. In Java version 7 and since, the `HashMap` implementation has been changed to use a `TreeMap` instead of a `LinkedList` to store values. In response to the discovery of the vulnerability, other languages such as Perl and Ruby have changed the hash function to `SipHash` [3] which mitigates hash-flooding vulnerabilities.

C. Detection and Prevention of DoS Attacks

Qie et al. [30] present a toolkit to make software that is robust against DoS attacks. This defensive approach prescribes annotating code where resources are used and released thus assisting in abuse detection and defensive action at runtime. SAFER [5] is a tool that detects semantic vulnerabilities in C programs that may be vulnerable to DoS attacks using malicious inputs. Both of these methods involve the analysis of code whereas our method is based on observing data on the stream. An approach that is closer to ours is JEP290 [20] implemented for Java deserialisation. JEP290 allows incoming serialisation data to be filtered based on patterns. These patterns can set limits on the depth of the graph, maximum number of internal references, size of the input data and the maximum array size allowed. It can also have patterns to match against class/package names to either block or allow the types permitted in the object stream. Approaches such as software compartmentalization for executing sensitive operations in controlled environments are well-known [13] [9]. Once such approach is described for Java deserialisation in [39]. Dietrich et al. [12] propose a language-level measure that uses code instrumentation to implement resource usage contracts on predetermined methods that are vulnerable (e.g. `toString`, `hashCode`.)

VIII. CONCLUSION

In this paper, we have discussed the study of multiple YAML libraries covering ten languages for DoS vulnerabilities and we have discovered seven previously unreported issues. The root of these vulnerabilities, YAML anchor/aliases is a major feature of the language which facilitates ease of hand-editing, readability and compactness. Protection mechanisms are available in the form of sandboxing, and code instrumentation. These mechanisms can be costly to deploy, and as an alternative we have proposed a light-weight filter that makes YAML safer while preserving the ability to use features such as aliases and deeply-nested structures. In future work, we intend to evaluate our filter on a wider variety of inputs.

¹⁵<https://bitbucket.org/unshorn/dosyamlpub/src/master/src/filter/>

ACKNOWLEDGMENTS

The authors would like to thank Long Qian and Shiyas Rasheed for their valuable feedback.

REFERENCES

- [1] Mehmod Abliz. Internet denial of service attacks and defense mechanisms. *University of Pittsburgh, Department of Computer Science, Technical Report*, 2011.
- [2] Efficient denial of service attacks on web application platforms. <https://perma.cc/Y55U-3PLR>, 2011. [Online; accessed 11-March-2019].
- [3] Jean-Philippe Aumasson and Daniel J Bernstein. SipHash: a fast short-input PRF. 2012.
- [4] Working with AWS CloudFormation Templates. <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/template-guide.html>, 2019. [Online; accessed 11-March-2019].
- [5] Richard Chang, Guofei Jiang, Franjo Ivancic, Sriram Sankaranarayanan, and Vitaly Shmatikov. Inputs of coma: Static detection of denial-of-service vulnerabilities. In *Proceedings CSF'09*, pages 186–199. IEEE, 2009.
- [6] Wouter Coekaerts. SerialDOS. <https://gist.github.com/coekie/a27cc406fc9f3dc7a70d>, 2015. [Online; accessed 15-January-2019].
- [7] CVE-2003-1564 (Billion Laughs). <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1564>, 2003. [Online; accessed 15-January-2019].
- [8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009.
- [9] Stefano Cristalli, Mattia Pagnozzi, Mariano Graziano, Andrea Lanzi, and Davide Balzarotti. Micro-virtualization memory tracing to detect and prevent spraying attacks. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 431–446, Austin, TX, 2016. USENIX Association.
- [10] Scott A Crosby and Dan S Wallach. Denial of Service via Algorithmic Complexity Attacks. In *Proceedings of 21th Usenix Security Symposium*, volume 2, 2003.
- [11] Johannes Dahse, Nikolai Krein, and Thorsten Holz. Code Reuse Attacks in PHP: Automated POP Chain Generation. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, CCS '14*, pages 42–53, New York, NY, USA, 2014. ACM.
- [12] Jens Dietrich, Kamil Jezek, Shawn Rasheed, Amjed Tahir, and Alex Potanin. Evil Pickles: DoS Attacks Based on Object-Graph Engineering. In *ECOOOP*, 2017.
- [13] Aristide Fattori, Andrea Lanzi, Davide Balzarotti, and Engin Kirda. Hypervisor-based malware protection with accessminer. *Comput. Secur.*, 52(C):33–50, July 2015.
- [14] YAML support for the Go language. <https://github.com/go-yaml/yaml>, 2019. [Online; accessed 11-March-2019].
- [15] Introducing JSON. <https://json.org>, 2019. [Online; accessed 15-January-2019].
- [16] JavaScript YAML parser and dumper. <https://github.com/nodeca/js-yaml>, 2019. [Online; accessed 11-March-2019].
- [17] LibYAML - A C library for parsing and emitting YAML. <https://github.com/yaml/libyaml>, 2019. [Online; accessed 11-March-2019].
- [18] F. Momot, S. Bratus, S. M. Hallberg, and M. L. Patterson. The seven turrets of babel: A taxonomy of langsec errors and how to expunge them. In *2016 IEEE Cybersecurity Development (SecDev)*, pages 45–52, Nov 2016.
- [19] OpenAPI Specification. <https://github.com/OAI/OpenAPI-Specification>, 2019. [Online; accessed 11-March-2019].
- [20] JEP 290: Filter Incoming Serialization Data. <http://openjdk.java.net/jeps/290>, 2016. [Online; accessed 11-March-2019].
- [21] YAML::Syck and JSON::Syck. <https://github.com/toddr/YAML-Syck>, 2019. [Online; accessed 11-March-2019].
- [22] Perl binding to libyaml. <https://github.com/ingydotnet/yaml-libyaml-pm>, 2019. [Online; accessed 11-March-2019].
- [23] PHP Internals. <http://www.phpinternalsbook.com/>, 2019. [Online; accessed 11-March-2019].
- [24] PHP YAML Extension. <https://pecl.php.net/package/yaml>, 2019. [Online; accessed 11-March-2019].
- [25] CVE-2014-9130 scanner.c in LibYAML 0.1.5 and 0.1.6, as used in YAML-XS module for Perl, allows denial of service. <https://www.cvedetails.com/cve/CVE-2014-9130/>, 2014. [Online; accessed 11-March-2019].
- [26] CVE-2017-18342 PyYAML could execute arbitrary code. <https://nvd.nist.gov/vuln/detail/CVE-2017-18342>, 2019. [Online; accessed 11-March-2019].
- [27] CVE-2018-13043 Debian devscripts using unsafe YAML Loading. <https://nvd.nist.gov/vuln/detail/CVE-2018-13043>, 2019. [Online; accessed 11-March-2019].
- [28] Symfony Yaml component. <https://symfony.com/doc/current/components/yaml.html>, 2019. [Online; accessed 11-March-2019].
- [29] Canonical source repository for PyYAML. <https://github.com/yaml/pyyaml>, 2019. [Online; accessed 11-March-2019].
- [30] Xiaohu Qie, Ruoming Pang, and Larry Peterson. Defensive programming: Using an annotation toolkit to build dos-resistant software. *ACM SIGOPS Operating Systems Review*, 36(SI):45–60, 2002.
- [31] A pure Rust YAML implementation. <https://github.com/chyh1990/yaml-rust>, 2019. [Online; accessed 11-March-2019].
- [32] Strongly typed YAML library for Rust. <https://github.com/dtolnay/serde-yaml>, 2019. [Online; accessed 11-March-2019].
- [33] snakeyaml: YAML 1.1 parser and emitter for Java. <https://bitbucket.org/asomov/snakeyaml>, 2019. [Online; accessed 11-March-2019].
- [34] Christopher Späth, Christian Mainka, Vladislav Mladenov, and Jörg Schwenk. SoK: XML Parser Vulnerabilities. In *Proceedings WOOT'16. USENIX*, 2016.
- [35] Bryan Sullivan. Security Briefs - XML Denial of Service Attacks and Defenses. In *MSDN Magazine Blog*, volume 24, 2009.
- [36] Arbitrary code execution via Swagger YAML parser (CVE-2017-1000207 and CVE-2017-1000208). <https://perma.cc/4PGB-LZBC>, 2017. [Online; accessed 11-March-2019].
- [37] Robert Tarjan. Depth first search and linear graph algorithms. *SIAM JOURNAL ON COMPUTING*, 1(2), 1972.
- [38] Rastislav Šrámek. *Uncertain Optimization Using Approximation Sets. An Algorithmic Prospective*. PhD thesis, ETH Zurich, 2013.
- [39] The deserialization problem. <https://perma.cc/H8KT-3TPA>, 2018. [Online; accessed 11-March-2019].
- [40] YAML Ain't Markup Language (YAML) Version 1.2. <https://yaml.org/spec/1.2/spec.html>, 2019. [Online; accessed 15-January-2019].